

L23: Exploratory Data Analysis and Distribution Fitting

Simulation Without a Black Box

Outline

Why Inputs Matter

EDA Checklist

CV as Shape Diagnostic

Histograms and ECDF

MLE with SciPy

AIC for Model Comparison

Overlay Fitted PDFs

Key Takeaways

Garbage In, Garbage Out

The GIGO Principle

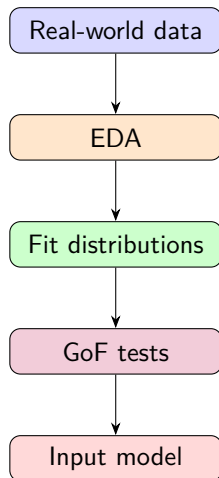
A simulation is an amplifier, not a corrector.

If interarrival times are mis-specified by 20%, mean queue length can be wrong by 60% or more near saturation ($\rho \rightarrow 1$).

Two distinct errors:

- **Wrong family** — Exponential when the true CV is 0.4
- **Wrong parameters** — Correct family, biased mean

Both distort every KPI the simulation produces.



EDA Checklist: What to Compute First

For any input sample of size n , compute in order:

1. n , min, max — sanity check for outliers / truncation
2. $\bar{x}$, s — location and spread
3. $\widehat{CV} = s/\bar{x}$ — shape diagnostic (*see next slide*)
4. Skewness $= \mu_3/s^3$ — asymmetry
5. Excess kurtosis $= \mu_4/s^4 - 3$ — tail weight
6. Histogram + ECDF — visual sanity

Warning

Always check $\min \geq 0$ for service times and interarrivals. Negative values signal a data cleaning problem.

```
import pandas as pd, scipy.stats as st
```

```
df = pd.read_csv("service_times.csv")  
x = df["service_min"].values
```

```
print(f"n      = {len(x)}")  
print(f"mean    = {x.mean():.3f}")  
print(f"std      = {x.std():.3f}")  
print(f"CV       = {x.std()/x.mean():.3f}")  
print(f"skew     = {st.skew(x):.3f}")  
print(f"kurt    = {st.kurtosis(x):.3f}")
```

CV: The Single Most Diagnostic Statistic

Definition

$CV = \sigma/\mu$ — the coefficient of variation measures variability relative to the mean, and is scale-free.

CV range	Likely family	Implication	M/M/1 error if ignored
≈ 1	Exponential	Memoryless arrivals	Baseline (no error)
< 1 (e.g. 0.3–0.7)	Erlang / Gamma	Regular service	Overestimates W_q
> 1 (e.g. 1.5–3)	Lognormal / Weibull	Heavy tail	Underestimates W_q
$\gg 1$	Pareto / heavy tail	Extreme events	Severely underestimates

Rule of thumb: If CV puts you in the wrong family, your queue-length forecasts may be off by a factor of 2–5 at high utilization.

Choosing Bin Width and Reading the ECDF

Freedman-Diaconis bin width:

$$h = 2 \cdot \text{IQR}(x) \cdot n^{-1/3}$$

- Adapts to spread; robust to outliers
- Preferred over Sturges ($\log_2 n + 1$) for skewed data

Why ECDF beats histogram for tails:

- No binning artifacts — every data point matters
- Tail behavior visible: does $1 - F_n(x)$ decay exponentially or as a power law?
- KS statistic is directly the max gap between ECDF and fitted CDF

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Freedman-Diaconis
```

```
iqr = np.percentile(x, 75) - \
      np.percentile(x, 25)
h = 2 * iqr * len(x)**(-1/3)
bins = int((x.max()-x.min())/h)
```

```
plt.hist(x, bins=bins, density=True,
         alpha=0.5, label="Histogram")
```

```
# ECDF
```

```
xs = np.sort(x)
ys = np.arange(1, len(x)+1)/len(x)
plt.figure()
plt.step(xs, ys, label="ECDF")
```

Maximum Likelihood Estimation with `scipy.stats`

MLE Principle

Choose parameters $\hat{\theta}$ that maximize the probability (likelihood) of observing the data $x_1, \dots, x_n$.

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n \log f(x_i; \theta)$$

Fix the location at zero (`floc=0`) for non-negative data — otherwise SciPy may shift the distribution below zero, which is physically impossible.

```
from scipy import stats

# Exponential
loc, scale = stats.expon.fit(x,
                             floc=0)

# Gamma (shape a, scale)
a, loc, scale = stats.gamma.fit(x,
                                 floc=0)

# Lognormal (s=sigma, scale=exp(mu))
s, loc, scale = stats.lognorm.fit(x,
                                   floc=0)

# Weibull minimum
c, loc, scale = stats.weibull_min.fit(
    x, floc=0)
```

AIC: Penalized Likelihood for Model Comparison

Akaike Information Criterion

$$\text{AIC} = -2\ell(\hat{\theta}) + 2k$$

where $\ell(\hat{\theta})$ is the maximized log-likelihood and k is the number of free parameters. **Lower AIC = better model.**

- Exponential: $k = 1$
- Gamma, Lognormal, Weibull: $k = 2$
- Penalty discourages over-fitting
- $\Delta\text{AIC} < 2$: essentially equivalent
- $\Delta\text{AIC} > 10$: strong evidence for lower-AIC model

Warning

AIC compares models on the *same data*. Do not compare across different sample sizes or after removing outliers from only one model.

```
def aic(dist, params, x):  
    ll = dist.logpdf(x, *params).sum()  
    k = len(params)  
    return -2*ll + 2*k  
  
candidates = {  
    "expon": stats.expon.fit(x,floc=0),  
    "gamma": stats.gamma.fit(x,floc=0),  
    "lognorm": stats.lognorm.fit(x,floc=0),  
    "weibull_min": stats.weibull_min.fit(  
        x,floc=0),  
    }  
  
for name,(dist_obj,params) in zip(  
    candidates.keys(),  
    [(getattr(stats,k),v)  
     for k,v in candidates.items()]):  
    print(name, aic(dist_obj,params,x))
```


Overlay Fitted PDFs on the Histogram

After fitting, always plot the fitted PDF over the empirical histogram:

```
xg = np.linspace(x.min(), x.max(), 300)
```

```
# After fitting gamma:
```

```
a, loc, sc = stats.gamma.fit(x, floc=0)
```

```
plt.hist(x, bins=bins, density=True,  
         alpha=0.4, label="Data")
```

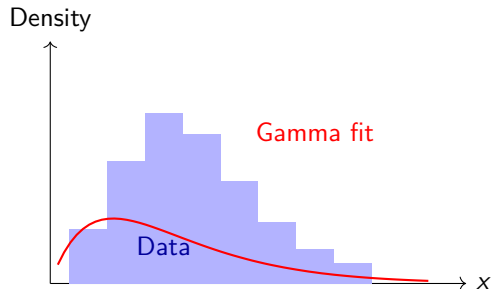
```
plt.plot(xg,  
         stats.gamma.pdf(xg, a, loc, sc),  
         label=f"Gamma(a={a:.2f})")
```

```
plt.legend()
```

```
plt.xlabel("Service time (min)")
```

```
plt.ylabel("Density")
```

Visual check catches multi-modality, heavy tails, or truncation that AIC alone may miss.



Key Takeaways

1. **GIGO**: mis-specified inputs distort every output KPI, especially near saturation.
2. **CV first**: a single ratio tells you which distributional family to investigate.
3. **Use ECDF alongside histogram** to expose tail behavior.
4. **Fix `fluc=0`** when fitting non-negative data in SciPy.
5. **AIC** penalizes complexity; prefer the lower-AIC model unless $\Delta\text{AIC} < 2$ or domain knowledge favors another.
6. **Always overlay the fitted PDF** — numbers alone can miss multi-modality and truncation.

Next Lecture

L24 — Goodness-of-fit testing: KS, chi-squared, and Q-Q plots.